

Departamento: **Engenharia Elétrica**

Professor Responsável: **Prof. Dr. Laerte Peotta de Melo**

Relatório do Laboratório 10

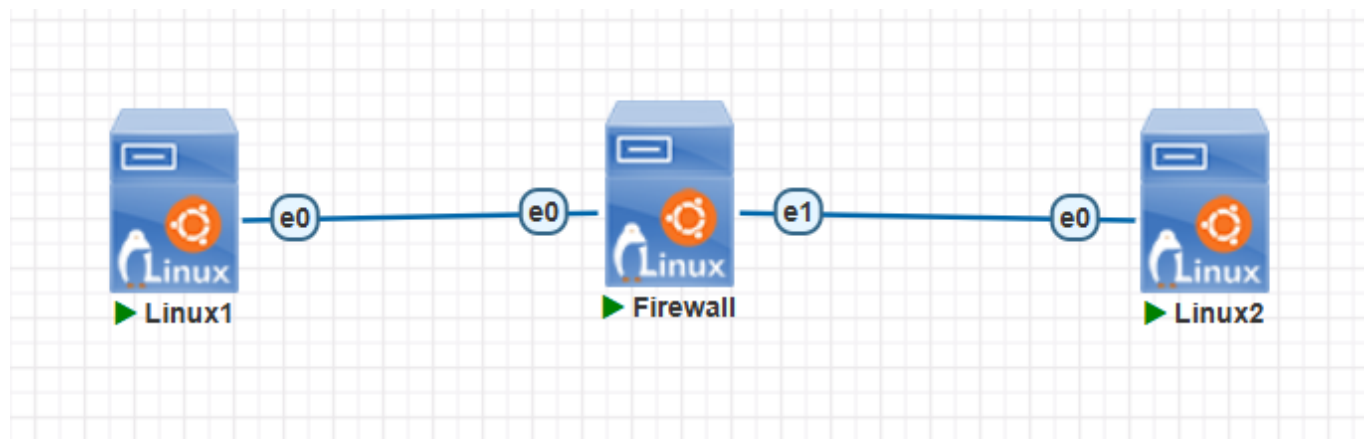
Identificação

- Nome: **Artur Kohara Guerra**
- Matrícula: **231025181**
- Turma: **01**

Objetivo

Implementar um **firewall de pacotes** em uma máquina Linux no PNetLab, posicionada entre **duas máquinas Linux básicas**, aplicando regras com **iptables** para controlar o tráfego entre duas redes distintas com base em endereço IP, protocolo e porta.

Topologia do Laboratório



O Linux 1 e o Linux 2 agem como clientes na comunicação e a máquina linux central age como firewall.

Procedimientos

Configuração dos hosts Linux

Configuração no PNetLab

```
Image: linux-tinycore-6.4
Ethernet: 1
```

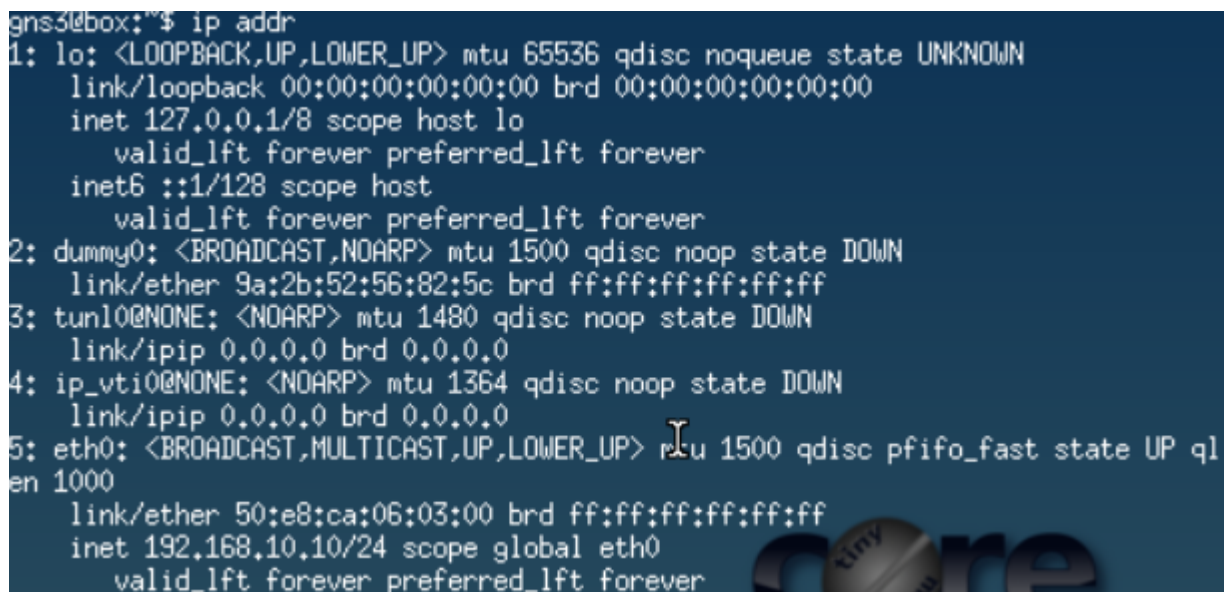
```
MTU: 1500
CPU: 1
RAM: 2048 MB
Console: VNC
Qemu Arch: x86_64
Qemu NIC: virtio-net-pci
TPM: Disabled
UEFI: desmarcado
```

Linux Cliente 1

Configuração do endereço IP e da rota padrão:

```
sudo ip addr add 192.168.10.10/24 dev eth0
sudo ip link set eth0 up
sudo ip route add default via 192.168.10.1
```

Verificando a configuração:

A terminal window showing the output of the 'ip addr' command. The output lists several network interfaces: 'lo' (loopback), 'dummy0', 'tunl0@NONE', 'ip_vti0@NONE', and 'eth0'. The 'eth0' interface is highlighted with a mouse cursor and shows the configuration: 'mtu 1500 qdisc pfifo_fast state UP qlen 1000', 'link/ether 50:e8:ca:06:03:00 brd ff:ff:ff:ff:ff:ff', and 'inet 192.168.10.10/24 scope global eth0'. A 'core' logo is visible in the bottom right corner of the terminal image.

```
gns3@box:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: dummy0: <BROADCAST,NOARP> mtu 1500 qdisc noop state DOWN
    link/ether 9a:2b:52:56:82:5c brd ff:ff:ff:ff:ff:ff
3: tunl0@NONE: <NOARP> mtu 1480 qdisc noop state DOWN
    link/ipip 0.0.0.0 brd 0.0.0.0
4: ip_vti0@NONE: <NOARP> mtu 1364 qdisc noop state DOWN
    link/ipip 0.0.0.0 brd 0.0.0.0
5: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP ql
    en 1000
    link/ether 50:e8:ca:06:03:00 brd ff:ff:ff:ff:ff:ff
    inet 192.168.10.10/24 scope global eth0
        valid_lft forever preferred_lft forever
```

Linux Cliente 2

Configuração do endereço IP e da rota padrão:

```
sudo ip addr add 192.168.20.10/24 dev eth0
sudo ip link set eth0 up
sudo ip route add default via 192.168.20.1
```

Verificando a configuração:

```
gns3@box:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: dummy0: <BROADCAST,NOARP> mtu 1500 qdisc noop state DOWN
    link/ether 22:11:18:a7:a9:7c brd ff:ff:ff:ff:ff:ff
3: tunl0@NONE: <NOARP> mtu 1480 qdisc noop state DOWN
    link/ipip 0.0.0.0 brd 0.0.0.0
4: ip_vti0@NONE: <NOARP> mtu 1364 qdisc noop state DOWN
    link/ipip 0.0.0.0 brd 0.0.0.0
5: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP ql
en 1000
    link/ether 50:08:c5:06:04:00 brd ff:ff:ff:ff:ff:ff
    inet 192.168.20.10/24 scope global eth0
        valid_lft forever preferred_lft forever
```

Configuração básica da máquina Linux Firewall

Configuração no PNetLab

```
Image: linux-ubuntu-24.04-server
Ethernet: 2
MTU: 1500
CPU: 2
RAM: 4096 MB
Console: VNC
Qemu Arch: x86_64
Qemu NIC: virtio-net-pci
TPM: Disabled
UEFI: desmarcado
Qemu options: -machine type=pc,accel=kvm -vga virtio -usbdevice tablet -boot
order=cd -cpu host -k pt-br
```

Configuração dos endereços IP das interfaces

```
sudo ip addr add 192.168.10.1/24 dev ens3
sudo ip addr add 192.168.20.1/24 dev ens4
sudo ip link set ens3 up
sudo ip link set ens4 up
```

Verificando a configuração:

```
user@ubuntu:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 50:7f:b5:05:fc:00 brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    inet 192.168.10.1/24 scope global ens3
        valid_lft forever preferred_lft forever
    inet6 fe80::527f:b5ff:fe05:fc00/64 scope link
        valid_lft forever preferred_lft forever
3: ens4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 50:7f:b5:05:fc:01 brd ff:ff:ff:ff:ff:ff
    altname enp0s4
    inet 192.168.20.1/24 scope global ens4
        valid_lft forever preferred_lft forever
    inet6 fe80::527f:b5ff:fe05:fc01/64 scope link
        valid_lft forever preferred_lft forever
```

Ativação do roteamento IP

```
sudo sysctl -w net.ipv4.ip_forward=1
```

Bloqueio de ping com `sysctl`

Nesta etapa, foi feito um teste simples para impedir que o firewall responda a requisições de ping.

```
sudo sysctl -w net.ipv4.icmp_echo_ignore_all=1
```

Teste:

```
gns3@box:~$ ping 192.168.10.1
PING 192.168.10.1 (192.168.10.1): 56 data bytes
^C
--- 192.168.10.1 ping statistics ---
5 packets transmitted, 0 packets received, 100% packet loss
gns3@box:~$ ping 192.168.20.1
PING 192.168.20.1 (192.168.20.1): 56 data bytes
^C
--- 192.168.20.1 ping statistics ---
7 packets transmitted, 0 packets received, 100% packet loss
```

Depois, reverteu-se a configuração.

```
sudo sysctl -w net.ipv4.icmp_echo_ignore_all=0
```

Teste inicial sem firewall

- Cliente 1

```
gns3@box:~$ ping 192.168.20.10
PING 192.168.20.10 (192.168.20.10): 56 data bytes
64 bytes from 192.168.20.10: seq=0 ttl=63 time=1.534 ms
64 bytes from 192.168.20.10: seq=1 ttl=63 time=1.796 ms
64 bytes from 192.168.20.10: seq=2 ttl=63 time=1.401 ms
64 bytes from 192.168.20.10: seq=3 ttl=63 time=1.355 ms
64 bytes from 192.168.20.10: seq=4 ttl=63 time=1.207 ms
^C
--- 192.168.20.10 ping statistics ---
5 packets transmitted, 5 packets received, 0% packet loss
round-trip min/avg/max = 1.207/1.458/1.796 ms
```

- Cliente 2

```
gns3@box:~$ ping 192.168.10.10
PING 192.168.10.10 (192.168.10.10): 56 data bytes
64 bytes from 192.168.10.10: seq=0 ttl=63 time=1.305 ms
64 bytes from 192.168.10.10: seq=1 ttl=63 time=1.471 ms
64 bytes from 192.168.10.10: seq=2 ttl=63 time=1.453 ms
64 bytes from 192.168.10.10: seq=3 ttl=63 time=1.265 ms
64 bytes from 192.168.10.10: seq=4 ttl=63 time=1.185 ms
^C
--- 192.168.10.10 ping statistics ---
5 packets transmitted, 5 packets received, 0% packet loss
round-trip min/avg/max = 1.185/1.335/1.471 ms
```

Limpeza das regras antigas

Antes de iniciar a configuração do firewall, limpou-se regras anteriores do **iptables**.

```
sudo iptables -F
sudo iptables -X
sudo iptables -Z
```

Definindo a política padrão da cadeia **FORWARD** como **DROP**. Dessa forma, qualquer outro tráfego não explicitamente permitido será bloqueado:

```
sudo iptables -P FORWARD DROP
```

Configuração das regras do firewall de pacotes

Nesta etapa, foi permitido apenas:

- **ICMP** do Linux Cliente 1 para o Linux Cliente 2;
 - **HTTP** do Linux Cliente 1 para o Linux Cliente 2;
 - tráfego de retorno correspondente a essas permissões, de forma explícita.
-

Permitindo ICMP entre os dois hosts

```
sudo iptables -A FORWARD -s 192.168.10.10 -d 192.168.20.10 -p icmp -j ACCEPT
sudo iptables -A FORWARD -s 192.168.20.10 -d 192.168.10.10 -p icmp -j ACCEPT
```

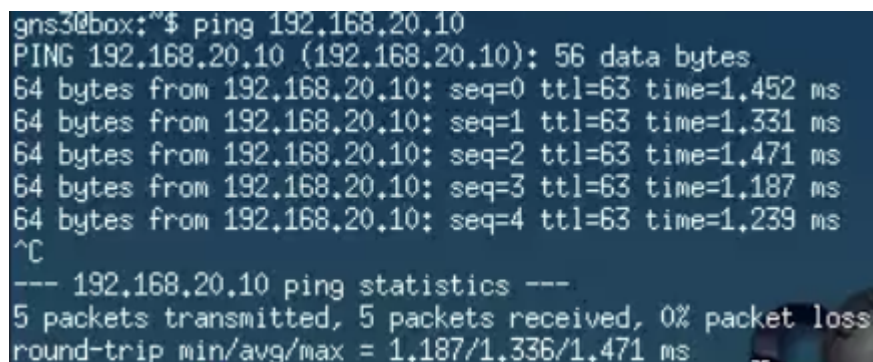
Permitindo HTTP do Cliente 1 para o Cliente 2

```
sudo iptables -A FORWARD -s 192.168.10.10 -d 192.168.20.10 -p tcp --dport 80 -j ACCEPT
sudo iptables -A FORWARD -s 192.168.20.10 -d 192.168.10.10 -p tcp --sport 80 -j ACCEPT
```

Testes práticos

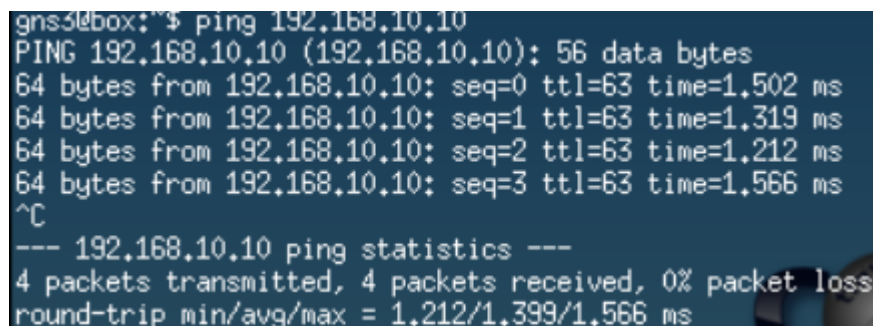
Teste de ICMP

- Linux Cliente 1



```
gns3@box:~$ ping 192.168.20.10
PING 192.168.20.10 (192.168.20.10): 56 data bytes
64 bytes from 192.168.20.10: seq=0 ttl=63 time=1.452 ms
64 bytes from 192.168.20.10: seq=1 ttl=63 time=1.331 ms
64 bytes from 192.168.20.10: seq=2 ttl=63 time=1.471 ms
64 bytes from 192.168.20.10: seq=3 ttl=63 time=1.187 ms
64 bytes from 192.168.20.10: seq=4 ttl=63 time=1.239 ms
^C
--- 192.168.20.10 ping statistics ---
5 packets transmitted, 5 packets received, 0% packet loss
round-trip min/avg/max = 1.187/1.336/1.471 ms
```

- Linux Cliente 2



```
gns3@box:~$ ping 192.168.10.10
PING 192.168.10.10 (192.168.10.10): 56 data bytes
64 bytes from 192.168.10.10: seq=0 ttl=63 time=1.502 ms
64 bytes from 192.168.10.10: seq=1 ttl=63 time=1.319 ms
64 bytes from 192.168.10.10: seq=2 ttl=63 time=1.212 ms
64 bytes from 192.168.10.10: seq=3 ttl=63 time=1.566 ms
^C
--- 192.168.10.10 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 1.212/1.399/1.566 ms
```

Teste de HTTP

No Cliente 2, utilizou-se o Netcat para simular um servidor HTTP na porta 80:

```
gns3@box:~$ sudo nc -l -p 80
GET / HTTP/1.1
Host: 192.168.20.10
User-Agent: Wget
Connection: close
```

No Linux Cliente 1, testou-se o acesso:

```
gns3@box:~$ wget -O- http://192.168.20.10
Connecting to 192.168.20.10 (192.168.20.10:80)
```

Dessa maneira, percebe-se que o teste foi um sucesso, pois a conexão foi estabelecida.

Teste de Telnet não permitido

No Linux Cliente 1, como o telnet não estava disponível, foi utilizado novamente o Netcat:

```
gns3@box:~$ nc -vz 192.168.20.10 23
nc: 192.168.20.10 (192.168.20.10:23): Connection timed out
```

Observa-se, portanto, que a conexão de fato falhou.

Teste de acesso iniciado pelo Cliente 2

No **Linux Cliente 2**, tentou-se iniciar conexão TCP para o Cliente 1 em uma porta qualquer com o Netcat.

```
gns3@box:~$ nc -vz 192.168.10.10 80
nc: 192.168.10.10 (192.168.10.10:80): Connection timed out
```

Logo, nota-se que a conexão falhou, pois não há regra permitindo esse tráfego.

Verificação das regras

- Regras com contadores

```
user@ubuntu:~$ sudo iptables -L -n -v
[sudo] password for user:
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in      out     source            destination
Chain FORWARD (policy DROP 13 packets, 780 bytes)
pkts bytes target      prot opt in      out     source            destination
 23  1932 ACCEPT      1    --  *      *       192.168.10.10     192.168.20.10
 23  1932 ACCEPT      1    --  *      *       192.168.20.10     192.168.10.10
 21  1392 ACCEPT      6    --  *      *       192.168.10.10     192.168.20.10      tcp dpt:80
 20   994 ACCEPT      6    --  *      *       192.168.20.10     192.168.10.10      tcp spt:80
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in      out     source            destination
```

- Regras da cadeia FORWARD


```
user@ubuntu:~$ sudo iptables -L FORWARD -n -v
Chain FORWARD (policy DROP 13 packets, 780 bytes)
pkts bytes target    prot opt in      out     source           destination
 23  1932 ACCEPT     1    --  *      *       192.168.10.10    192.168.20.10
 23  1932 ACCEPT     1    --  *      *       192.168.20.10    192.168.10.10
 21  1392 ACCEPT     6    --  *      *       192.168.10.10    192.168.20.10      tcp dpt:80
 20   994 ACCEPT     6    --  *      *       192.168.20.10    192.168.10.10      tcp spt:80
```

- Regras em formato detalhado

```
user@ubuntu:~$ sudo iptables -S
-P INPUT ACCEPT
-P FORWARD DROP
-P OUTPUT ACCEPT
-A FORWARD -s 192.168.10.10/32 -d 192.168.20.10/32 -p icmp -j ACCEPT
-A FORWARD -s 192.168.20.10/32 -d 192.168.10.10/32 -p icmp -j ACCEPT
-A FORWARD -s 192.168.10.10/32 -d 192.168.20.10/32 -p tcp -m tcp --dport 80 -j ACCEPT
-A FORWARD -s 192.168.20.10/32 -d 192.168.10.10/32 -p tcp -m tcp --sport 80 -j ACCEPT
```

Questões para análise

- **1. O que caracteriza um firewall de pacotes?**

Um firewall de pacotes analisa cada pacote de rede individualmente e decide se ele deve ser permitido ou bloqueado com base em informações presentes no cabeçalho do pacote, como endereço IP de origem e destino, protocolo e porta. Ele não acompanha o estado das conexões, tomando decisões apenas com base nas regras configuradas.

- **2. Quais campos do pacote foram usados nas regras deste laboratório?**

As regras utilizaram os campos de endereço IP de origem (**-s**), endereço IP de destino (**-d**), protocolo (**-p**), porta de destino (**--dport**) e porta de origem (**--sport**)

- **3. Por que foi necessário ativar o IP forwarding no Linux?**

Foi necessário ativar o IP forwarding para que o Linux pudesse encaminhar pacotes entre suas duas interfaces de rede. Sem essa funcionalidade, o sistema apenas receberia pacotes destinados a ele próprio, não atuando como roteador ou firewall entre as duas redes.

- **4. Qual é a função da cadeia **FORWARD** no **iptables**?**

A cadeia **FORWARD** é responsável por processar pacotes que atravessam o host, ou seja, pacotes que entram por uma interface e saem por outra. Em um firewall ou roteador Linux, essa cadeia controla quais comunicações entre redes diferentes serão permitidas ou bloqueadas.

- **5. Por que o tráfego não permitido foi bloqueado mesmo sem regras específicas para todos os protocolos?**

O tráfego não permitido foi bloqueado mesmo sem regras específicas, pois a política padrão da cadeia FORWARD foi configurada como DROP. Dessa forma, qualquer pacote que não correspondesse a uma regra explícita de permissão seria automaticamente descartado pelo firewall.

- **6. Qual a diferença entre permitir HTTP e permitir ICMP?**

Permitir HTTP significa autorizar conexões TCP destinadas à porta 80, utilizadas para serviços web. Já permitir ICMP significa autorizar mensagens de controle e diagnóstico da rede, como as utilizadas pelo

comando `ping`.

- **7. O que muda quando a política padrão da cadeia `FORWARD` é `DROP`?**

Quando a política padrão é `DROP`, todo tráfego é bloqueado por padrão. Apenas os pacotes que corresponderem a regras explícitas de permissão serão encaminhados. Essa abordagem aumenta a segurança ao seguir o princípio de negar tudo e permitir apenas o necessário.

- **8. Por que esse laboratório ainda não é considerado um firewall stateful?**

Esse laboratório ainda não é considerado um firewall stateful, pois as regras analisam cada pacote isoladamente e não acompanham o estado das conexões. Foi necessário criar regras específicas para o tráfego de ida e para o tráfego de retorno. Em um firewall stateful, o sistema reconhece conexões já estabelecidas e permite automaticamente os pacotes de resposta associados a elas.

- **9. Qual a importância da ordem das regras no `iptables`?**

O `iptables` processa as regras de cima para baixo. Quando um pacote corresponde a uma regra, a ação definida é executada e as regras seguintes não são analisadas. Por isso, a ordem das regras é fundamental para garantir que o tráfego seja tratado corretamente e que regras mais específicas não sejam sobrescritas por regras mais gerais.

- **10. Quais vantagens práticas surgem ao usar hosts Linux básicos no lugar de VPCs neste laboratório?**

O uso de hosts Linux permite realizar testes mais realistas, pois oferece ferramentas como `ping`, `wget`, `curl`, `nc`, servidores HTTP e outras aplicações de rede. Além disso, possibilita observar o comportamento dos serviços, analisar portas, gerar tráfego real e validar as regras do firewall de forma mais completa do que seria possível com VPCs simples.

- **11. Qual a diferença entre bloquear o ping com `sysctl` e bloquear ICMP com `iptables`?**

O parâmetro `sysctl` utilizado (`net.ipv4.icmp_echo_ignore_all`) impede que o próprio sistema operacional responda a requisições de ping destinadas ao firewall. Já o `iptables` permite controlar o tráfego ICMP que atravessa ou chega ao sistema com muito mais flexibilidade, podendo filtrar pacotes com base em origem, destino, interfaces e outros critérios. Assim, o `sysctl` altera o comportamento do host, enquanto o `iptables` atua como mecanismo de filtragem de pacotes.

Conclusão

Neste laboratório foi possível implementar e testar um firewall de pacotes utilizando o `iptables` em uma máquina Linux atuando como intermediária entre duas redes distintas. Foram configurados o endereçamento IP, o encaminhamento de pacotes por meio do IP forwarding e regras de filtragem baseadas em endereço IP, protocolo e porta. Os testes realizados demonstraram o funcionamento correto das permissões para tráfego ICMP e HTTP, bem como o bloqueio de conexões não autorizadas. Dessa forma, o laboratório permitiu compreender na prática como um firewall de pacotes controla o tráfego entre redes e como políticas de segurança baseadas no princípio de "negar por padrão e permitir apenas o necessário" contribuem para aumentar a segurança da infraestrutura de rede.